FleetZero

FleetPulse & BusPulse Development Portfolio

Role: Full-Stack Developer (Part-Time)
Duration: Jan. 2026 – Apr. 2026
Company: QDM Creative | Toronto, ON | Remote

Technologies: Angular 19, TypeScript, HTML, CSS, Angular Material, RxJS, Karma, Jasmine, ApexCharts, REST APIs, Git, CI/CD

Prepared by
Jaturaput (Mac) Jongsubcharoen

Telephone: +1 (647) 425-9756
Email: macker.jong@gmail.com
LinkedIn: www.linkedin.com/in/jaturaput-jongsubcharoen
GitHub: https://github.com/Jaturaput-Jongsubcharoen

Project Overview

As a Full-Stack Developer at FleetZero, I contributed to the development of FleetPulse and BusPulse, two enterprise fleet management platforms used to monitor vehicles, facilities, inspections, and maintenance operations.

My work included developing new dashboard features, integrating REST APIs, implementing interactive data visualizations, optimizing application performance, fixing production bugs, improving responsive user interfaces, and writing comprehensive Karma/Jasmine unit tests to maintain code quality and support the CI/CD pipeline.

Throughout the project, I collaborated with senior developers and product stakeholders using Git-based workflows, code reviews, and Agile development practices to deliver production-ready features.

Project Objectives

The project aimed to:

- Develop scalable fleet management dashboards for administrators and clients.
- Visualize fleet, inspection, and maintenance data through interactive widgets.
- Integrate frontend components with backend REST APIs.
- Improve dashboard performance and user experience.
- Maintain high code quality through automated testing and CI/CD validation.
- Deliver responsive and reliable enterprise web applications.

My Contributions

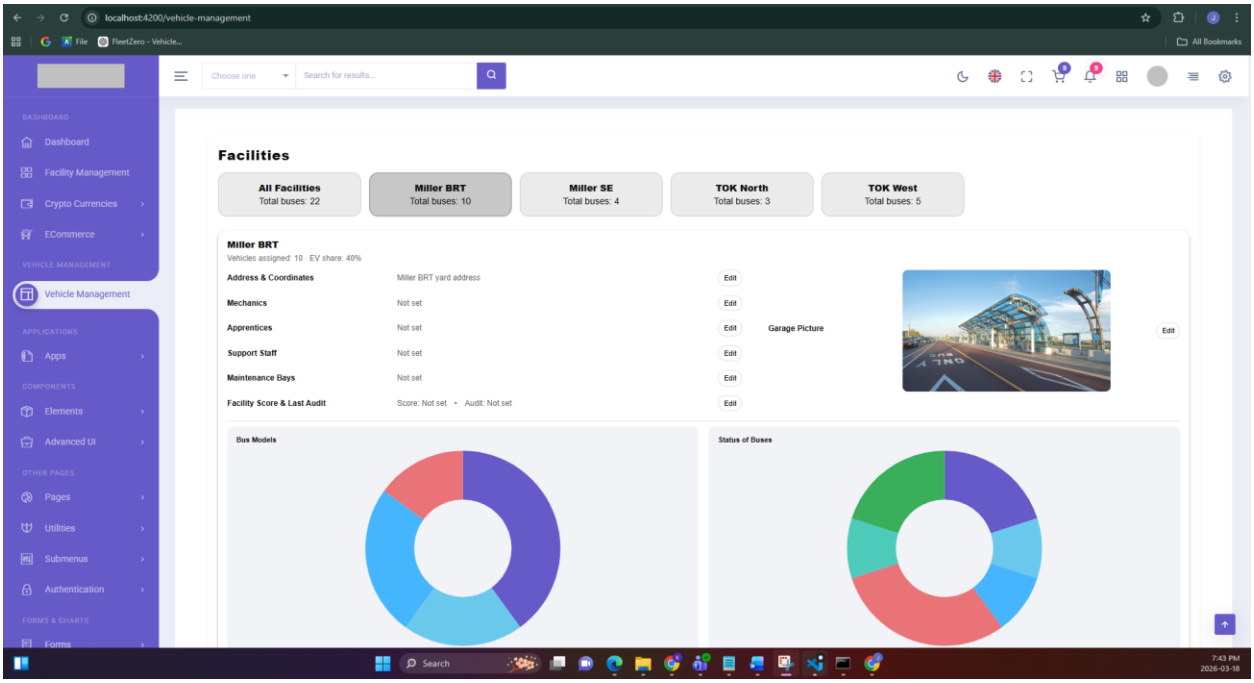
During my time at FleetZero, I contributed across multiple areas of the application, including:

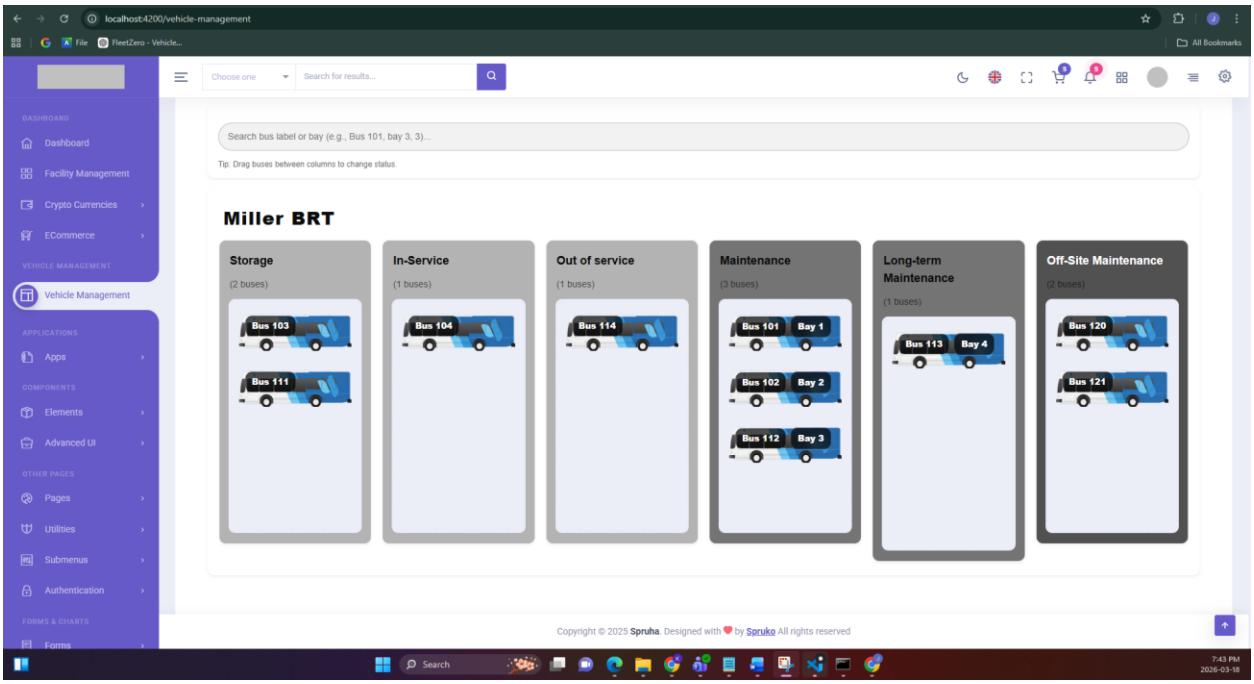
- Frontend development using Angular 19 and TypeScript.
- Dashboard design and implementation.
- REST API integration.
- Performance optimization.
- Bug fixing and dependency management.
- Interactive chart development using ApexCharts.
- Vehicle tracking and timeline visualization.
- Responsive UI improvements.
- Unit testing with Karma and Jasmine.
- CI/CD pipeline maintenance and test validation.

The following sections demonstrate the major features and technical contributions I completed throughout the project.

1. Working on the FleetPulse project

1.1. Setup FleetPulse Angular UI (initial code and assets)





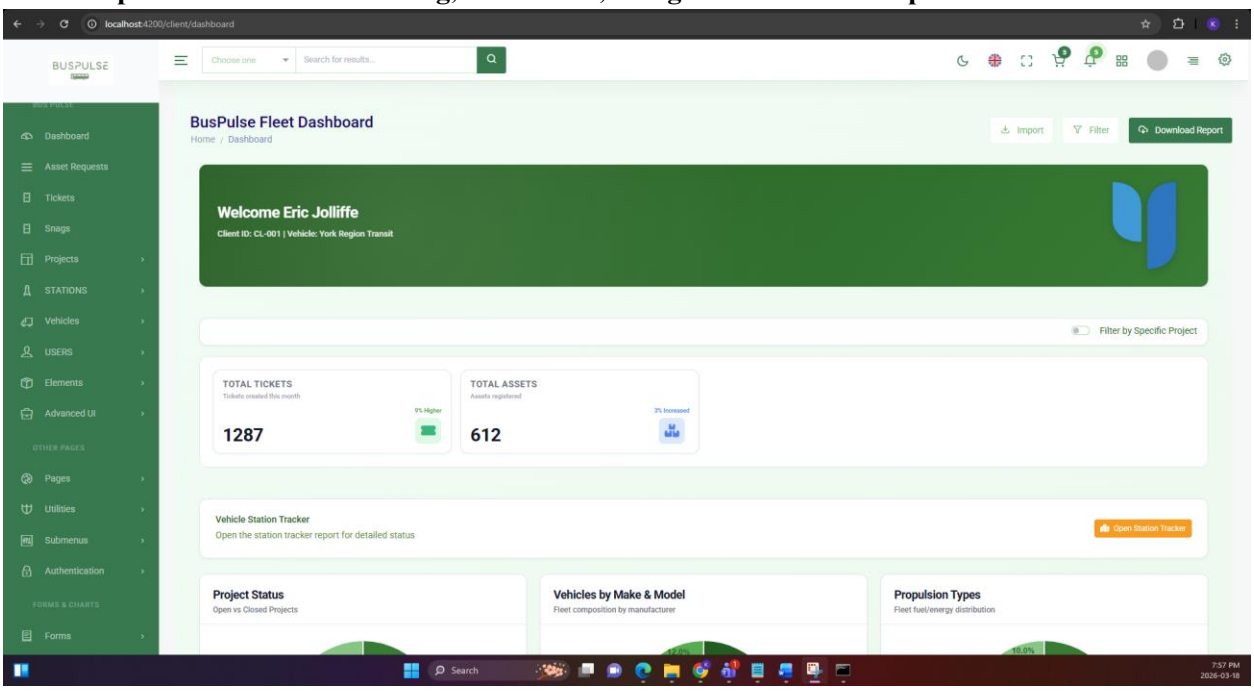
I set up the initial FleetPulse Angular UI based on the Spruha template, including:

- Dashboard layout components
- Facility & garage UI components
- Drag & scroll support for columns
- Facility assets & garage images
- Initial environment + styles setup
- implement drag & drop feature with the theme mode

1.2. Add unit tests for Vehicle Management UI components to make to ensure the code has no error.
Adds Karma/Jasmine unit tests for the UI components in the Facilities folder involved in the Vehicle Management dashboard Included: VehicleManagementComponent (drag/drop, search, search result, bay modal) and Facilities folder like FacilityButtons, FacilityColumns, FacilityBay, FacilitySearch, FacilitySearchResult, FacilityStrips (using mocked Chart.js component)

2. Working on BusPulse project "before" combining it into one file

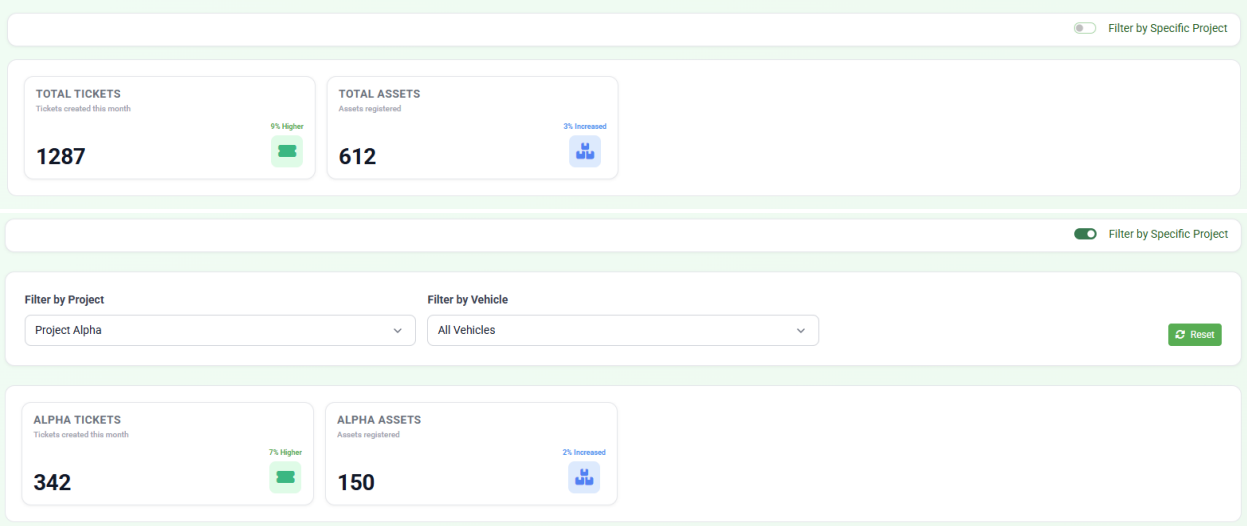
2.1. Set up Client UI: /client routing, dashboard, navigation and client reports



I add the initial Client UI under /client, including routing, dashboard, client data, and auth/nav wiring. Use the same widget from Shared Reuse (Unmodified – Direct Use from shared/). Client UI reuses existing shared modules/components without modifying their code:

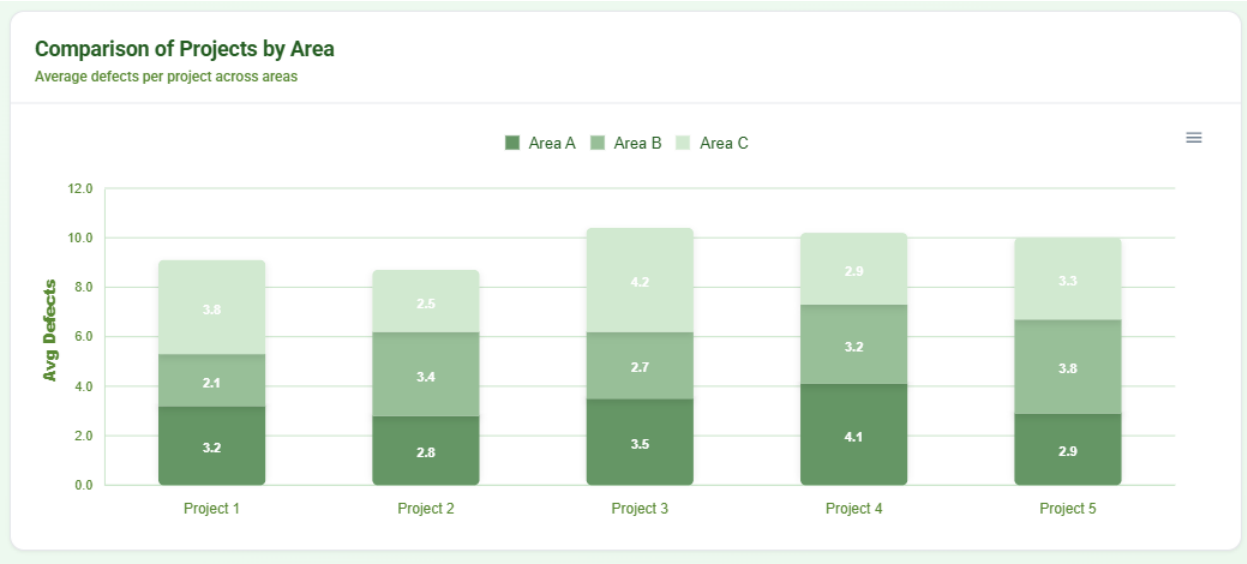
- Layouts: full-layout, content-layout, page-header, header, footer, sidemenu, switcher, notification-sidebar
- Charts: 6 shared dashboard chart components
- Reports: 6 shared report components and report services
- Auth/Guards: auth.guard, roleGuard
- Navigation service: nav.service menus and role-based routing
- All reused as-is from shared/ without changes.

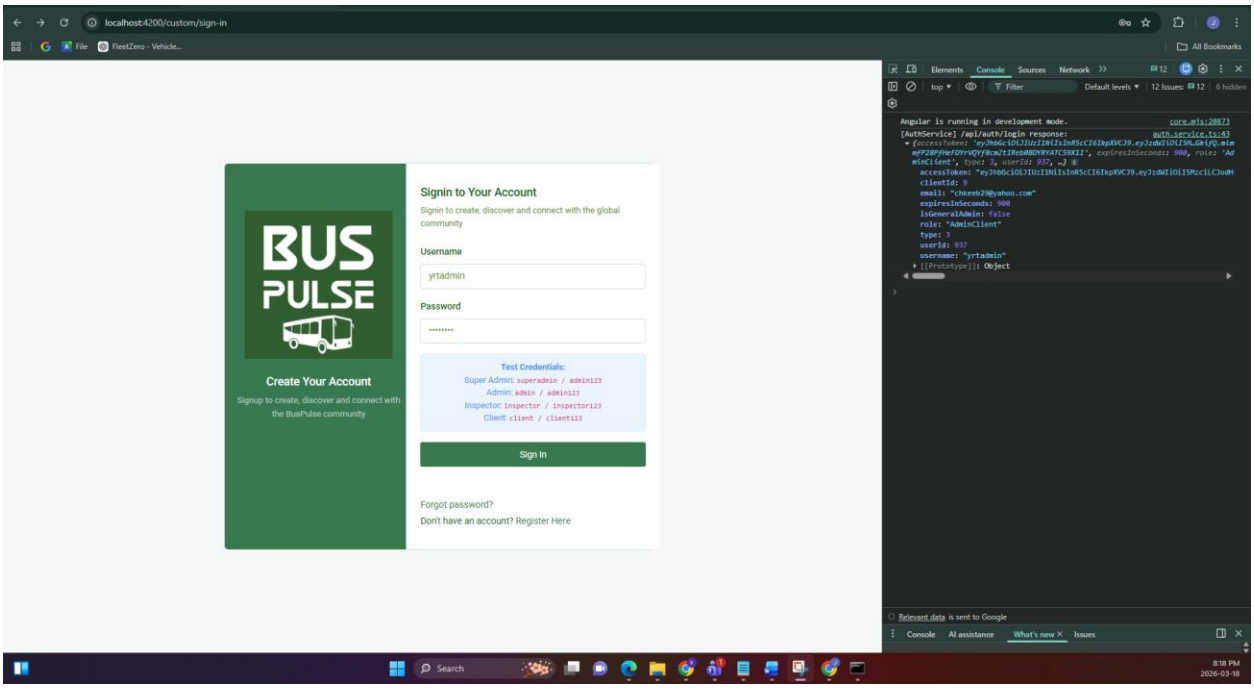
2.2. Hide charts for individual project & UI improvements.



Fixed ApexCharts console errors when toggling "Filter by Specific Project". Added complete theme support (light/dark mode) for client dashboard. Improved chart styling and widget visibility for filtered views.

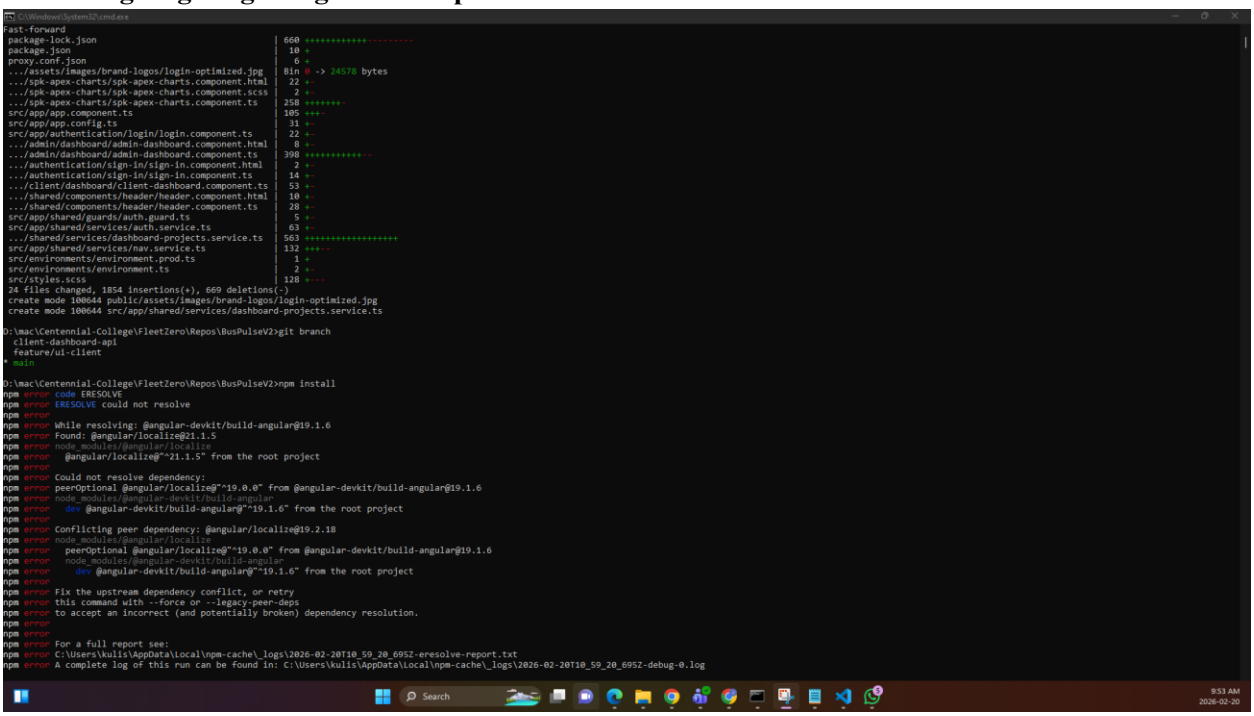
2.3. Enhance Client dashboard API integration and UI



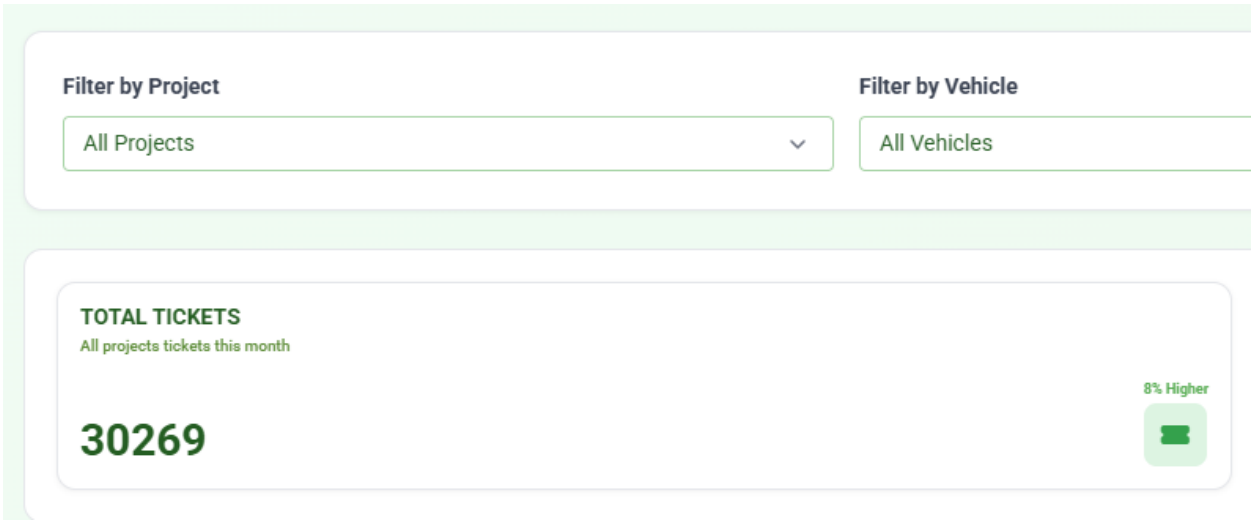


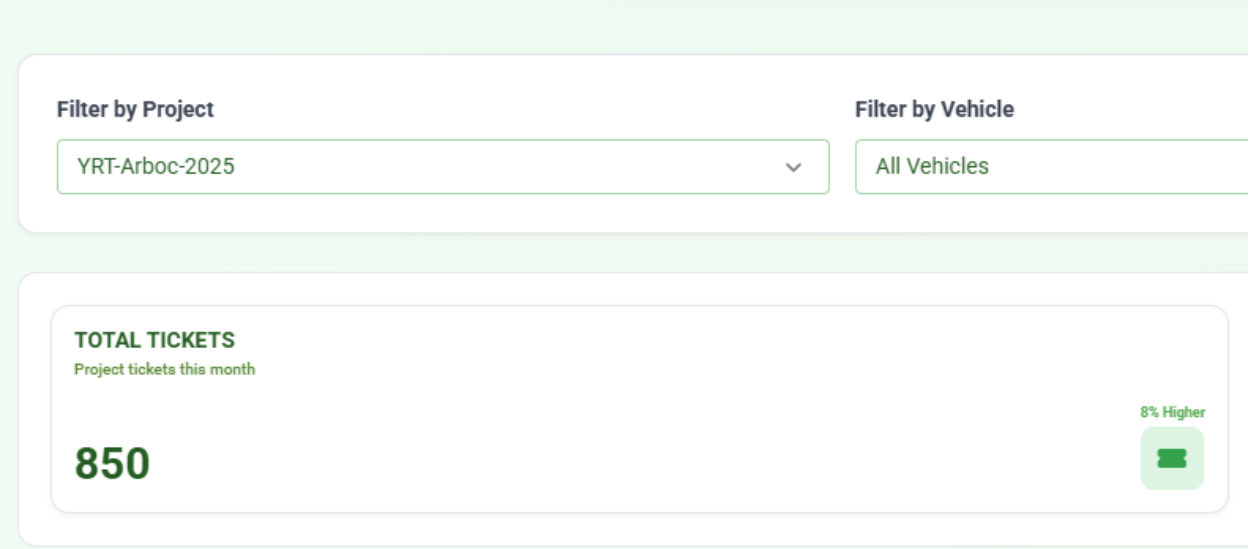
Changed background logo of “Bus Pulse Cloud” from dark green to transparent for better visual consistency. Enabled always-on data labels for the stacked bar chart in “Comparison of Projects by Area” so values display without hover (tooltips remain functional).

2.4. Fixing Bug: align Angular 19 dependencies



Fix Angular dependency alignment (Angular 19 compatibility) After pulling latest main code from Adiya's code. Removed @kolkov/angular-editor (Angular 20+ peer conflict). Pinned ng-apexcharts to 1.15.0 (Angular 19 compatible). Aligned @angular/material, @angular/cdk, @angular/localize to 19.2.18





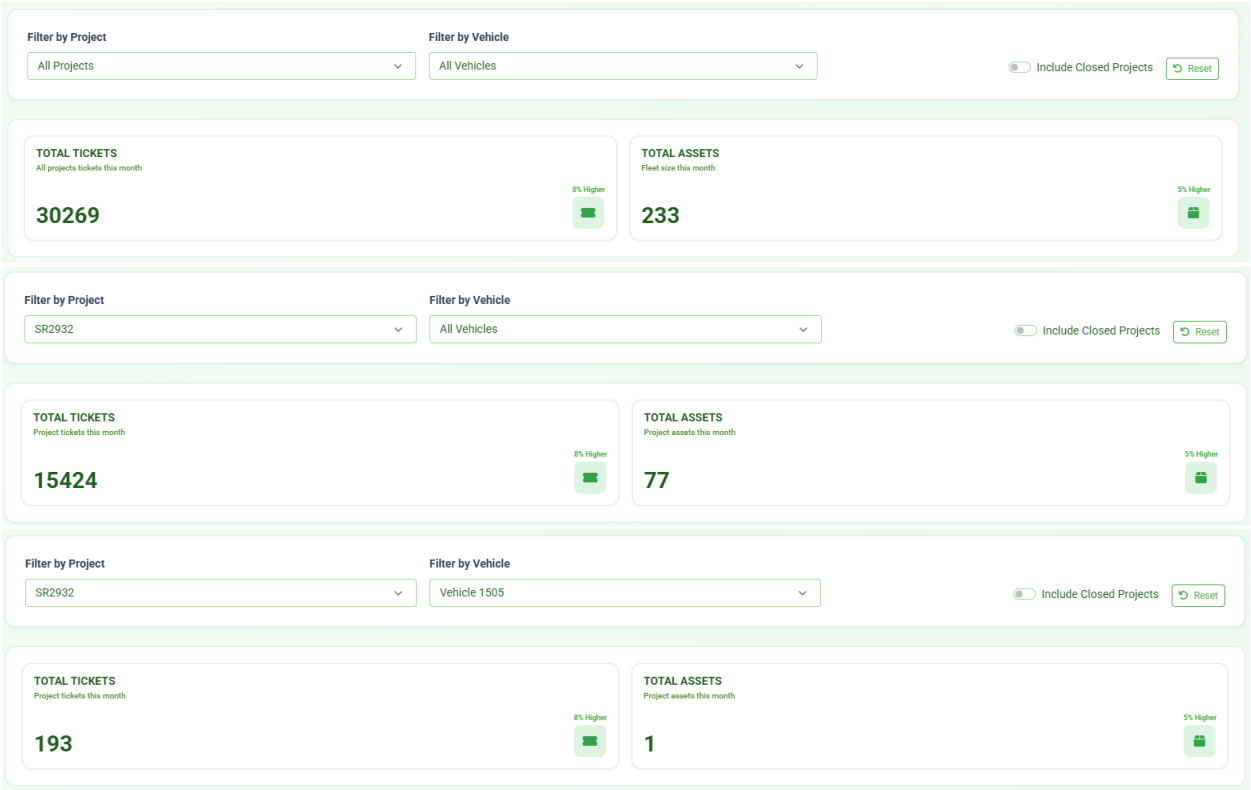
2.5. Use shared dashboard ticket APIs for project/vehicle-filtered totals

Updated Client Dashboard to use shared /api/tickets/dashboard for ticket totals based on active filters “project + all vehicles => project total tickets” and “project + specific vehicle => project + vehicle ticket total”. Integrated /api/projects/{projectId}/vehicles to dynamically load fleetNumbers per selected project. In the UI, increase the thickness of the Project Status, Propulsion Types, Repeated Defects, and Safety Critical Defects widgets to improve readability. Also, update the color scheme to use green shades.

2.6. Show treemap values by default for overall and repeated defects by area

In "Overall Defects by Area", display values for each vehicle inspection area by default without requiring hover interaction as Suraj requested. In "Repeated Defects by Area", display values for each vehicle inspection area by default without requiring hover interaction as Suraj requested.

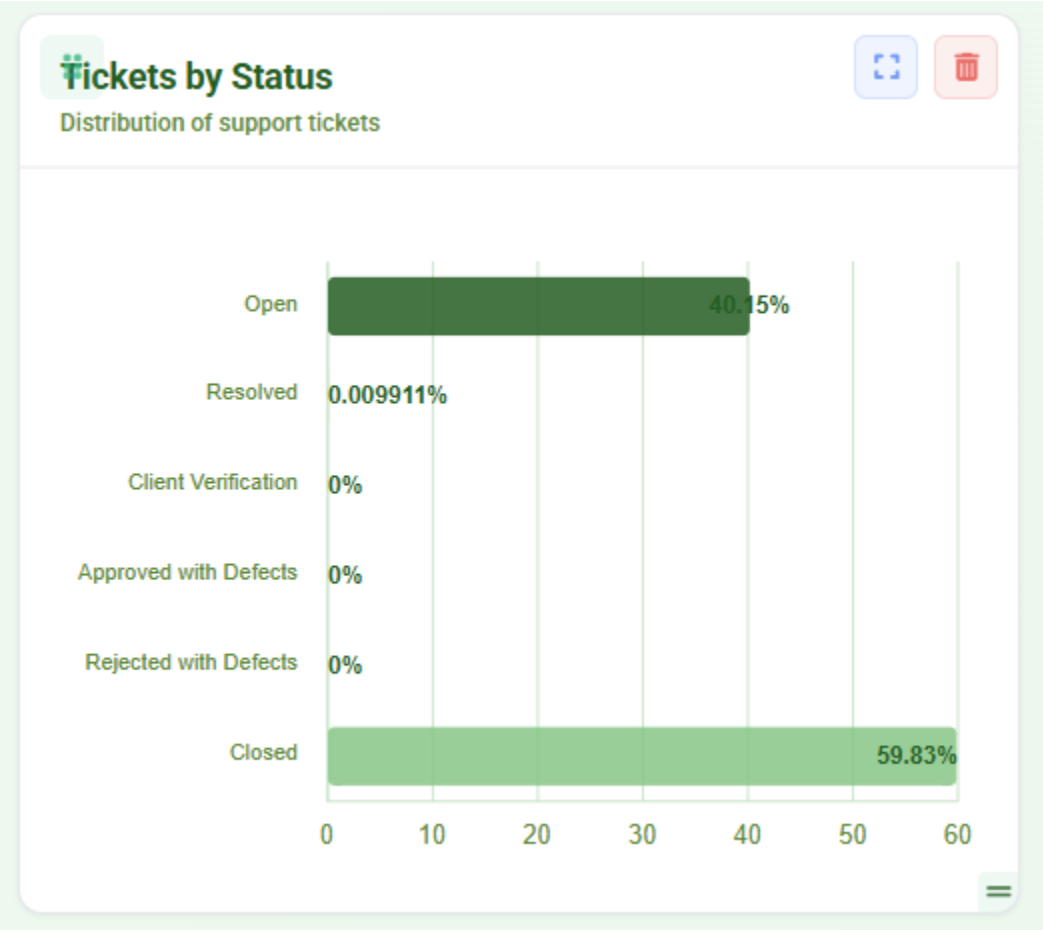
2.7. Deriving total tickets and total assets, as combining everything into a single file causes the API fetching to break.



Updated dashboard.component.ts, making refreshClientView() now requests ticket totals for project/vehicle combinations (single-project, all-projects aggregation, and vehicle-scoped queries). Total Assets computed from vehicle lists:

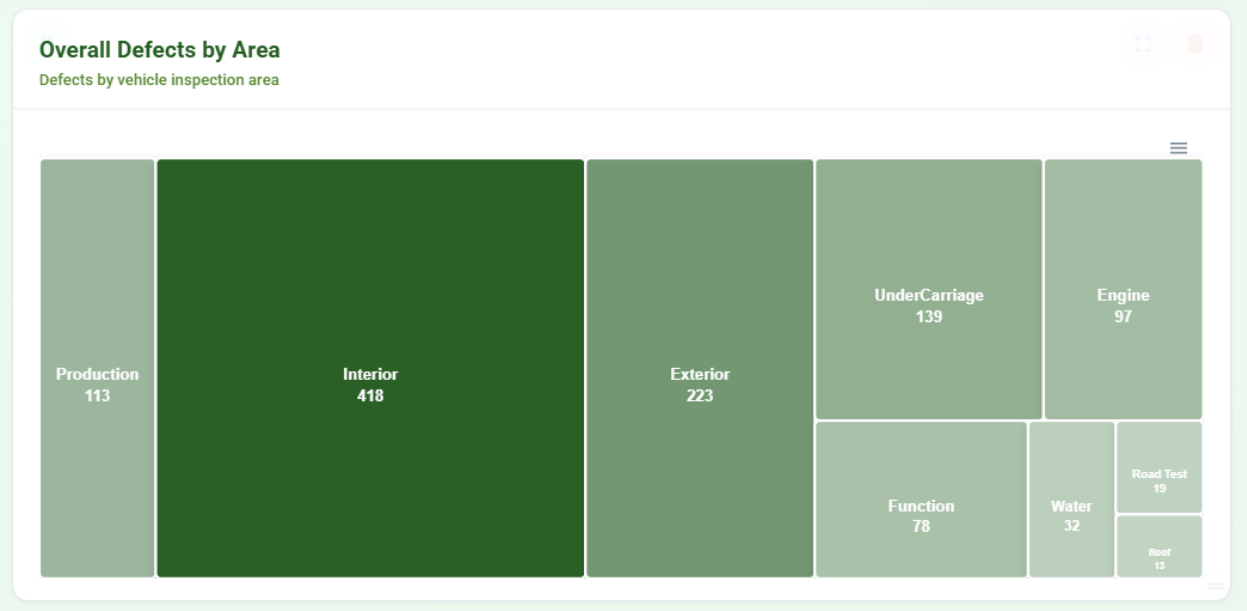
1. Specific vehicle -> 1
2. Specific project + All Vehicles -> project vehicle count (uses totalVehiclesCount or derived length)
3. All Projects + All Vehicles -> allClientVehicles.length
4. All Projects + specific vehicle -> 1

2.8. feat(dashboard): integrate Tickets by Status widget with API, canonicalize status labels, and unify client/admin dashboards (refs #450)



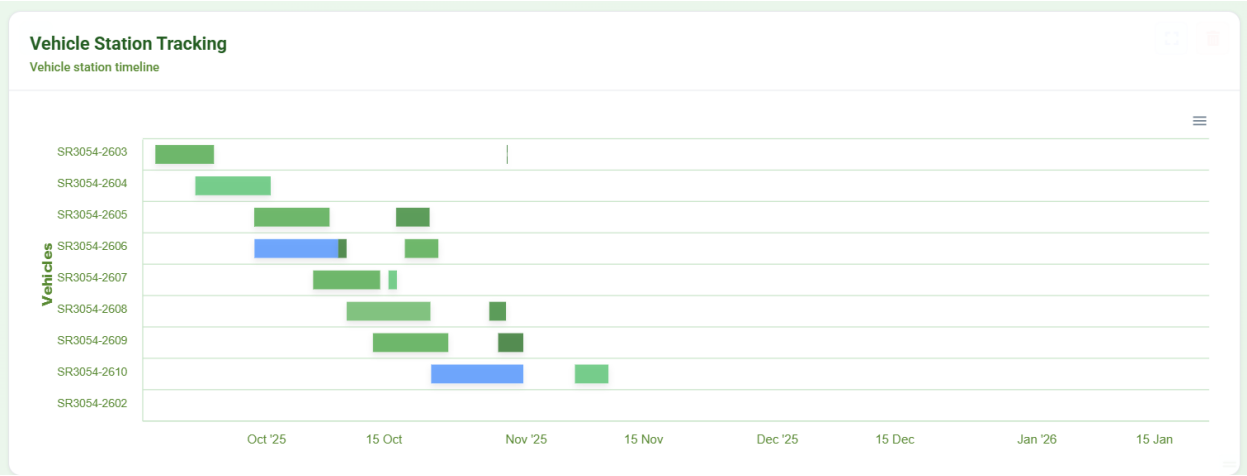
Replace demo or hardcoded Tickets by Status data with authoritative values retrieved from the /api/tickets/dashboard endpoint. Implement status normalization, percentage distribution calculation, and tooltip enhancements for the chart visualization. This update ensures the widget displays consistent ticket status data across both client and admin dashboards.

2.9. Widget: implement defects-by-area treemap with API + Production placeholder



Connected the treemap widget to the existing API endpoint: GET /api/tickets/dashboard. The widget now uses the overallByArea field returned by the API to display Category Inspection areas such as Interior, Exterior, Engine, Roof, Function, Water, Road Test, and UnderCarriage.
Each area is rendered as a block in the treemap where the block size represents the number of defects.
Production Placeholder: The dashboard design also requires "Production" to appear as one additional block in the treemap. At that time, the API does not return Production defect counts. To allow visualization of the expected layout, a temporary placeholder (~10%) for Production has been implemented.

2.10. Vehicle Station Tracking Widget: Render Fix, Performance Optimization, and UX Enhancements



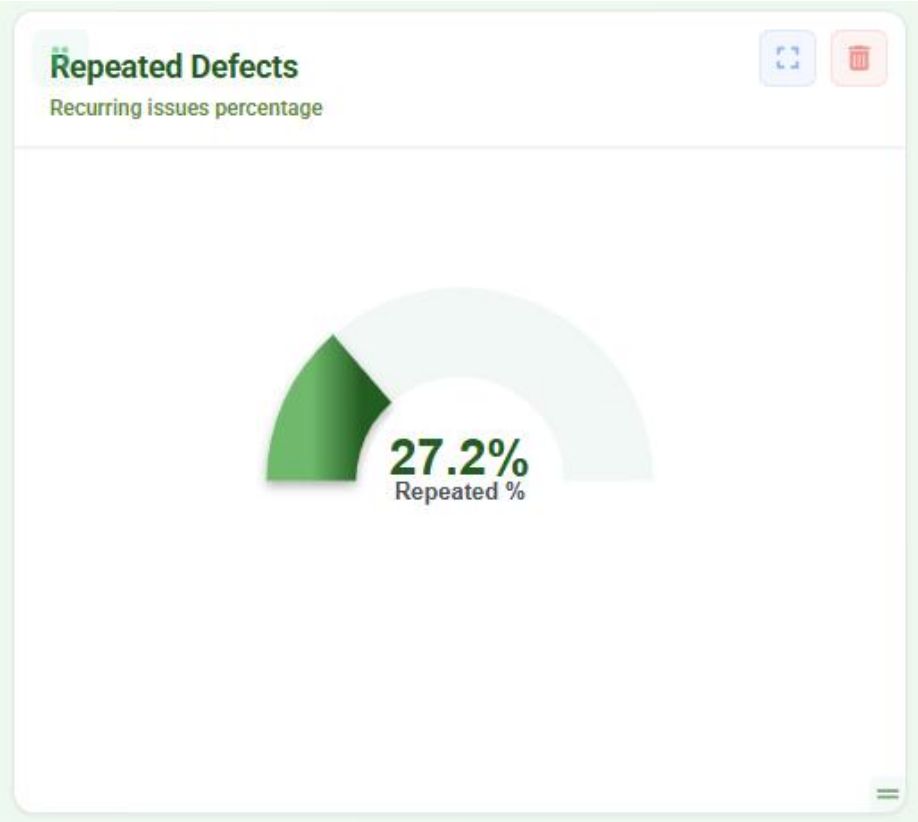
Implemented rendering fixes and usability improvements for the Vehicle Station Tracking widget. Key updates include:
- Restored widget rendering after recent code merges and ensured compile compatibility.
- improved tooltip information when hovering over timeline bars:

- Display Station Name instead of the previous “Station” label.
- Show Station Number when available.
- Include formatted Start and End date-times.

- improved chart readability:
  - Expanded X-axis (Date) and Y-axis (Vehicle) layout spacing.
  - Adjusted chart sizing to reduce overlap when many vehicles are displayed.
- Added horizontal and vertical scrolling for the widget so large timelines remain navigable while keeping vehicle labels visible.

These updates improve clarity, usability, and stability of the Vehicle Station Tracking widget across dashboard views.

2.11. Fix Client Dashboard widget display logic and Repeated Defects gauge behavior



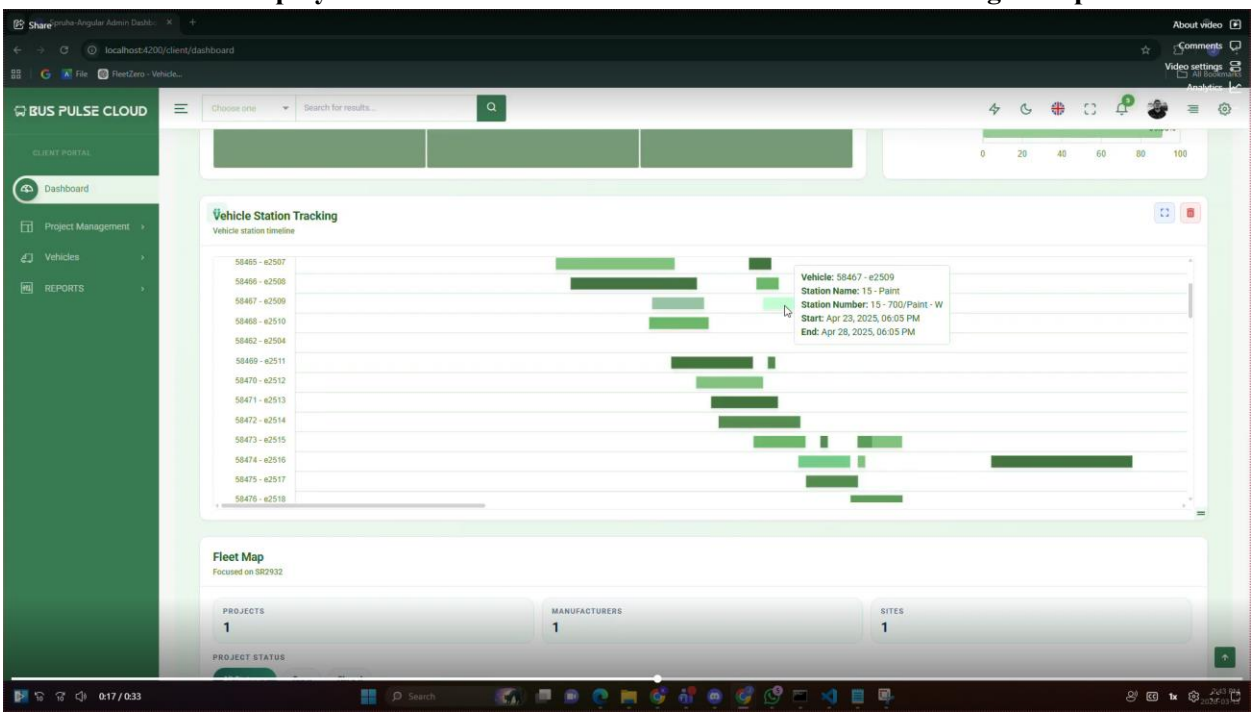
1. Widget Display Logic in both client and admin dashboard

Updated the Client Dashboard behavior so the "Vehicle Station Tracking" widget is hidden when the default filters are: “Filter by Project = All Projects” and “ Filter by Vehicle = All Vehicles”

2. Repeated Defects Widget Incorrect Value in both client and admin dashboard

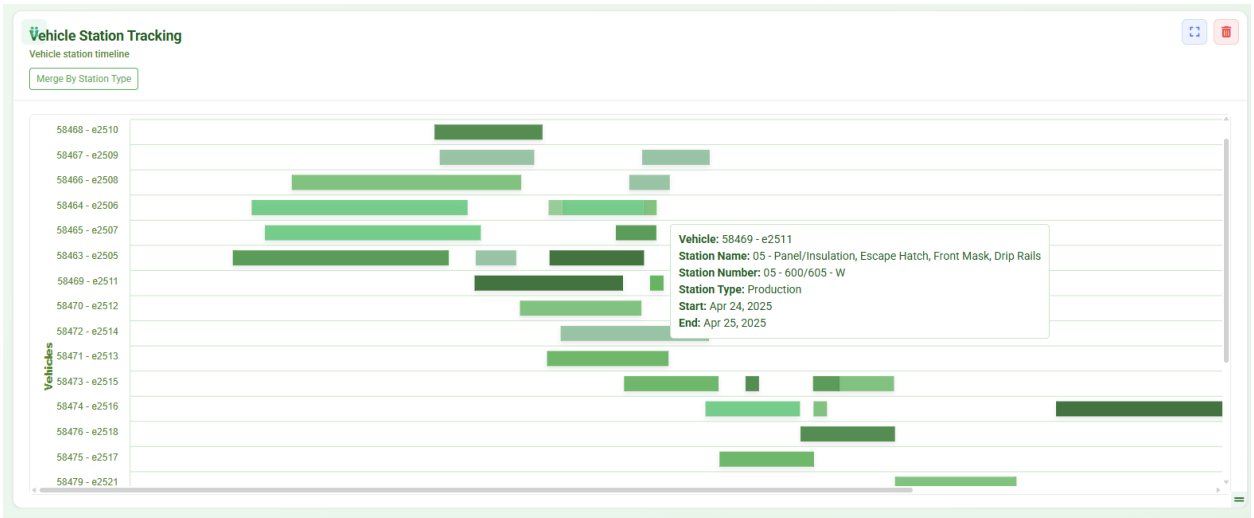
Improved the Repeated Defects percent resolution logic by adding normalizePercentValue() to handle ratio-style values returned by the API, adding repeatedTickets reader handling, using repeatedPercent when available and valid, computing percent from repeatedTickets / totalTickets when repeatedPercent is missing or unreliable, and using weighted calculation for array responses when needed.

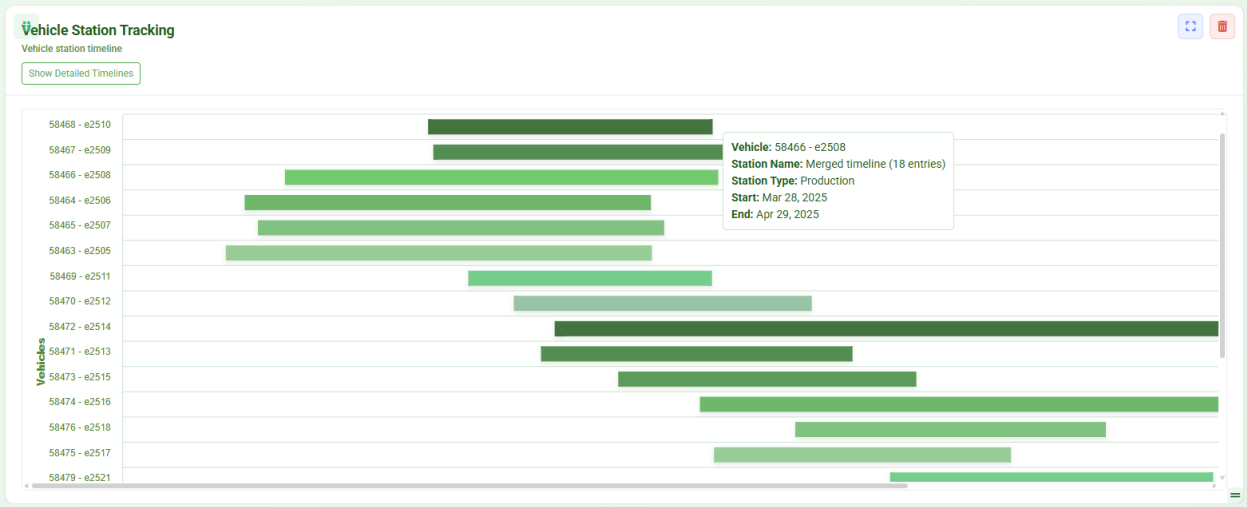
2.12. Remove time display from Start and End dates in Vehicle Station Tracking tooltip



Removed time from startDate and endDate in Vehicle Station Tracking tooltip. Tooltip now shows date only. Merged latest main into this branch before PR.

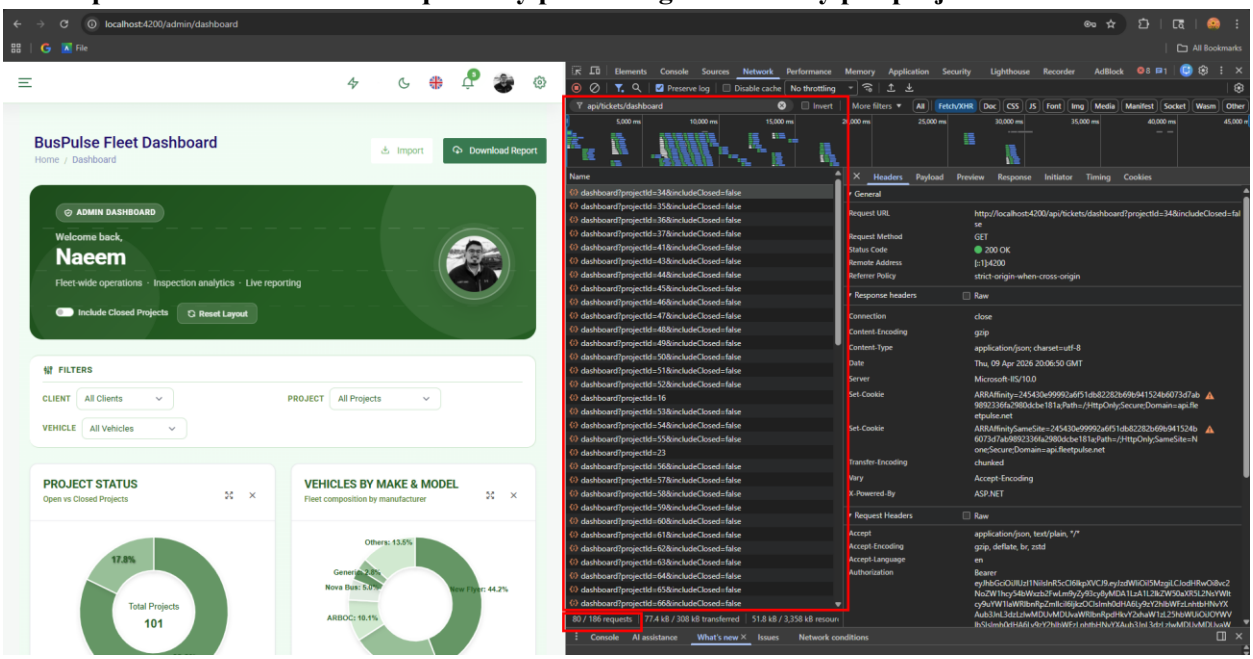
2.13. Display correct stationTypeName in Vehicle Station Tracking widget based on stationTypeId





I improved the Vehicle Station Tracking timeline parsing for more reliable timestamps and added a toggle to switch between detailed view (individual entries) and merged view (grouped by stationTypeName). I also added hover details to display station type and timeline values. Additionally, "None" from the API is treated as a valid station type, so it appears as a separate group in merged view.

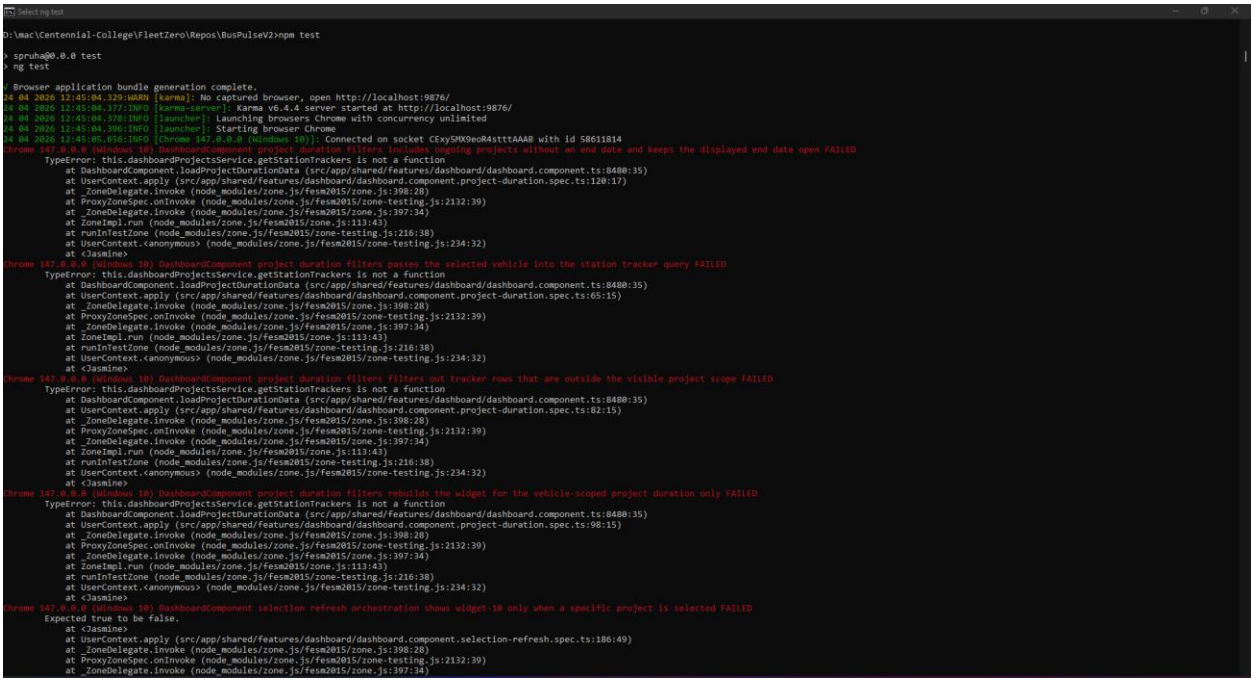
2.14. Optimize dashboard API requests by preventing unnecessary per-project calls

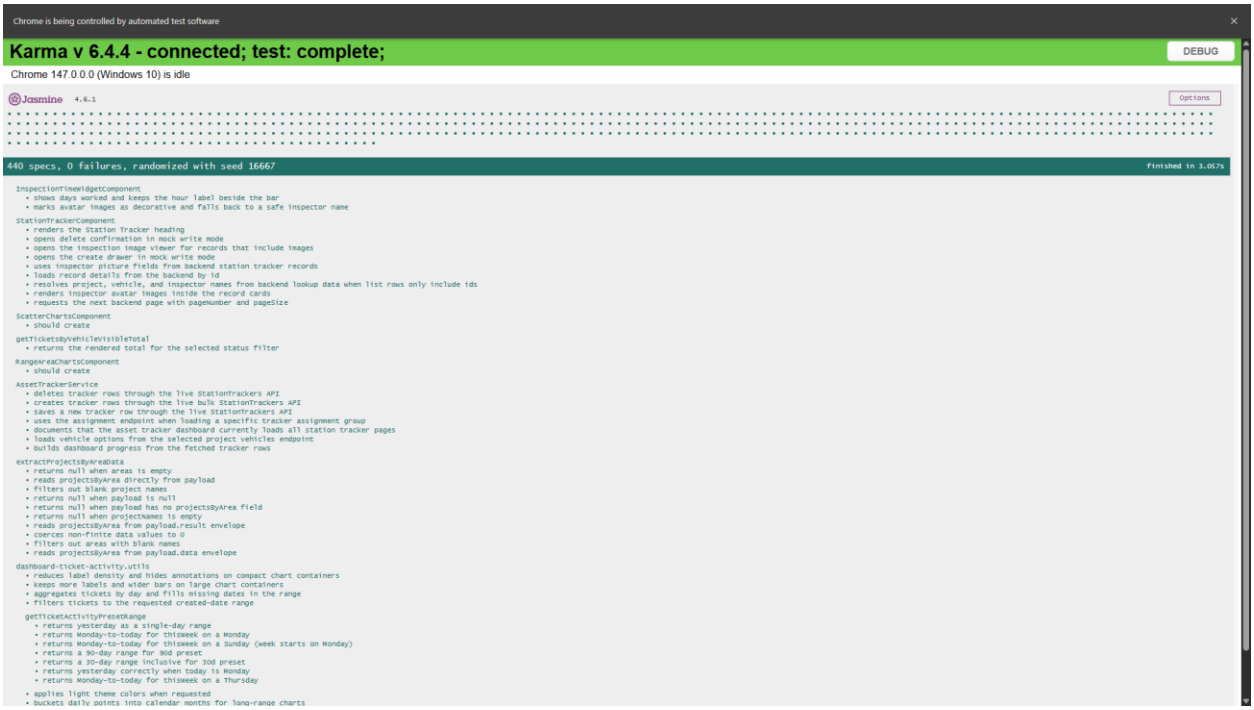


Before the optimization, when the dashboard loaded with the default filters (**All Projects** and **All Vehicles**), it triggered around **80 requests (out of 186 total)** to the /api/tickets/dashboard endpoint. The dashboard was making separate API requests for every project in the background even though no specific project was selected, resulting in unnecessary network traffic and slower page loading.
Updated the dashboard request logic so that when the project filter is set to **All Projects**, it skips the per-project request loop and exits early instead of fetching data for every project individually. Applied the same optimization across related dashboard components to ensure project-specific API calls are only made when a user explicitly selects a project.

These changes eliminated the repeated API requests during the initial dashboard load while preserving the existing behavior for project-specific filtering. The optimization significantly reduced unnecessary network traffic and improved dashboard performance and responsiveness.

2.15. Add Karma/Jasmine unit tests for Station Tracker enhancements





Added new Karma/Jasmine unit tests for the recently implemented Station Tracker enhancements to ensure the feature passes the **CI/CD pipeline** requirements. Updated the corresponding spec files using

mocked services and API responses without modifying production code, covering timeline rendering, inspector profile image handling, pagination, API data processing, chart utilities, and dashboard helper functions. The test suite completed successfully with **440 passing, 0 failing** tests and is ready for PR review.